

BANKING SYSTEM USING C++

- **PROJECT TITLE**

Banking System – Customer, Account and Transaction Management

- **ABSTRACT**

The Banking System is a C++ based application developed to simulate basic banking operations. The system allows users to create customer accounts, deposit and withdraw money, transfer funds between accounts, and maintain transaction records. It provides a simple and efficient way to manage banking activities while demonstrating the practical implementation of Object-Oriented Programming concepts such as classes, objects, encapsulation, and data management.

- **INTRODUCTION**

Banks play an important role in managing financial transactions and customer accounts. With the advancement of technology, banking operations have become computerized to improve accuracy, security, and efficiency. This project aims to develop a simple Banking System using C++ that can perform essential banking functions. The system helps users manage account information, perform transactions, and view transaction history in an organized manner.

- **PROBLEM STATEMENT**

Managing customer accounts and transactions manually can lead to errors, data inconsistency, and inefficiency. There is a need for a computerized system that can securely manage customer information, account balances, and transaction records while providing quick access to banking services.

- **OBJECTIVES**

- To create and manage customer accounts.
- To maintain account information and balances.
- To provide deposit and withdrawal facilities.
- To enable fund transfer between accounts.
- To store and display transaction history.
- To demonstrate Object-Oriented Programming concepts in C++.

- **SCOPE OF THE PROJECT**

The Banking System is designed for educational purposes and small-scale banking operations. It focuses on customer management, account handling, transaction processing, and account information display. The project can be further enhanced to support advanced banking features.

- **SYSTEM REQUIREMENTS**

Hardware Requirements:

- Computer or Laptop
- Minimum 4 GB RAM
- Keyboard and Monitor

Software Requirements:

- Operating System: Windows/Linux
- C++ Compiler (GCC, CodeBlocks, Turbo C++, VS Code)
- Standard C++ Library

- **METHODOLOGY**

The project follows an Object-Oriented Programming approach. Separate classes are created for Customer, Account, and Transaction. These classes interact with one another to perform banking operations. User input is accepted through a menu-driven interface, and all transactions are recorded and displayed when required.

- **CLASS DESIGN**

- **Customer Class**

Attributes:

- Customer ID
- Customer Name

Functions:

- Create Customer
- Display Customer Information

Account Class

Attributes:

- Account Number
- Account Balance

Functions:

- Deposit Money

- Withdraw Money
- Transfer Funds
- Display Balance

Transaction Class

Attributes:

- Transaction Type
- Transaction Amount

Functions:

- Store Transaction Details
- Display Transaction History

• FEATURES OF THE SYSTEM

- Customer Account Creation
- Account Management
- Deposit Operations
- Withdrawal Operations
- Fund Transfer Facility
- Transaction Recording
- Transaction History Display
- Balance Inquiry
- Account Information Display

- **WORKING OF THE SYSTEM**

1. The user creates a customer account.
2. An account number is assigned to the customer.
3. The customer can deposit money into the account.
4. The customer can withdraw money if sufficient balance is available.
5. Funds can be transferred from one account to another.
6. Every transaction is recorded in the transaction history.
7. The customer can view account details and recent transactions.

- **ALGORITHM**

Step 1: Start the program.

Step 2: Create customer and account.

Step 3: Accept customer details.

Step 4: Display menu options.

Step 5: Select a banking operation.

Step 6: Perform the selected operation.

Step 7: Update account balance.

Step 8: Store transaction details.

Step 9: Display account information and transaction history.

Step 10: Exit the program.

ADVANTAGES

- Simple and user-friendly.

- Reduces manual calculation errors.
- Provides quick access to account information.
- Maintains transaction records efficiently.
- Demonstrates real-world banking operations.

LIMITATIONS

- Data is stored temporarily during program execution.
- No user authentication mechanism.
- No database connectivity.
- Limited to basic banking functions.

FUTURE ENHANCEMENTS

- Integration with databases.
- Secure login and authentication.
- Online banking support.
- Loan management system.
- Interest calculation feature.
- Account statement generation.

• APPLICATIONS

- Educational projects.
- Banking management simulations.
- Learning Object-Oriented Programming.
- Academic and internship submissions.

- **RESULT**

The Banking System successfully performs account creation, deposits, withdrawals, fund transfers, and transaction management. The system maintains account information accurately and provides an efficient way to manage banking operations.

- **CONCLUSION**

The Banking System developed using C++ provides a simple and effective solution for managing customer accounts and banking transactions. The project demonstrates the practical implementation of Object-Oriented Programming concepts and helps users understand how banking systems operate in real-world environments.